

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(национальный исследовательский университет)

**Отчет**

**О выполнении лабораторной работы №3**

**«Глубокие рекуррентные нейронные сети»**

**По дисциплине «Нейросетевые технологии искусственного интеллекта»**

**Вариант задания №2**

Выполнила студентка группы М8О-109СВ-25

Бондарева Е.Е. \_\_\_\_\_

Проверил и принял

Доцент каф. 802 Леонов С.С. \_\_\_\_\_

С оценкой \_\_\_\_\_

## Лабораторная работа №3

### Сценарий работы

**Этап 1.** Построение рекуррентной нейронной сети для распознавания эмоциональной окраски отзывов из базы данных IMDB.

Целью данного этапа лабораторной работы является создание рекуррентного бинарного классификатора эмоциональной окраски отзывов из набора данных IMDB. Пошаговая реализация поставленной цели включает:

1. Загрузку набора данных IMDB;
2. Создание векторного представления слов из набора данных;
3. Разделение данных на обучающий и тестовый наборы;
4. Подготовку данных для передачи в нейронную сеть;
5. Конструирование сети: создание сети из слоя Embedding и рекуррентных слоев заданного типа в соответствии с вариантом;
6. Настройку оптимизатора с выбором функции потерь и метрики качества. Число эпох принять от 30 до 60;
7. Проведение проверки решения, выделяя контрольное множество;
8. Вывод графиков функции потерь и точности;
9. Использование обученной сети для предсказания на новых данных;
10. Сопоставление полученных результатов с одномерной сверточной сетью (три сверточных слоя).

Таблица 1.1 Вариант задания для первого этапа

| № | Количество рекуррентных слоев | Размерность векторного представления | Количество нейронов на слое | Рекуррентный слой |
|---|-------------------------------|--------------------------------------|-----------------------------|-------------------|
| 2 | 1                             | 32                                   | 64                          | LSTM              |

**Этап 2.** Построение прогноза температуры с использованием рекуррентных и сверточных нейронных сетей.

Целью данного этапа лабораторной работы является создание нейронной сети, реализующей прогноз температуры по набору данных Jena Climate. Пошаговая реализация поставленной цели включает:

1. Загрузку набора данных Jena Climate;
2. Разделение данных на обучающий и тестовый наборы;
3. Создание генератора данных, выбрав базу данных за 10 дней, задержку в 1 день, 1 значение в час и размер пакета из варианта;
4. Подготовку данных для передачи в нейронную сеть;
5. Выполнить конструирование сети, создание сети рекуррентных слоев заданного типа в соответствии с вариантом;
6. Настройку оптимизатора с выбором функции потерь и метрики качества. Число эпох принять от 30 до 60;
7. Проведение проверки решения, выделяя контрольное множество;
8. Вывод графиков функции потерь и точности;
9. Использование обученной сети для предсказания на новых данных;
10. Сопоставление полученных результатов с сетью, первый блок которого является одномерной сверточной сетью (три сверточных слоя), а затем идет второй блок, указанный в варианте.

Таблица 1.2 Вариант задания для второго этапа

| № | Количество рекуррентных слоев | Количество нейронов на слое | Рекуррентный слой | Прореживание |
|---|-------------------------------|-----------------------------|-------------------|--------------|
| 2 | 2                             | 32-64                       | GRU               | 0.1          |

## Этап 1. Построение сверточной нейронной сети для распознавания объектов из базы данных MNIST.

Импорт библиотек:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import ssl
```

На первом этапе выполнения лабораторной работы были инициализированы основные константы, управляющие процессом подготовки данных и обучения модели.

```
words_count, maxlen = 8000, 500
train_size, test_size = 10000, 1000
epochs, batch_size = 30, 128
```

Загрузка и преобработка данных:

Следующим шагом выполняется загрузка набора данных, который будет использоваться для обучения и тестирования модели. В данном случае используется популярный датасет IMDB, содержащий 50 000 размеченных киноподборок, где каждая метка (0 или 1) соответствует отрицательному или положительному отзыву соответственно.

```
ssl._create_default_https_context = ssl._create_unverified_context
(x_train, y_train), (x_test, y_test) =
keras.datasets.imdb.load_data(num_words=words_count)
ssl._create_default_https_context = ssl.create_default_context
```

Далее выполняется стандартизация длины текстовых последовательностей для подготовки данных к подаче в нейронную сеть. Исходные отзывы имеют переменное количество слов, что несовместимо с архитектурой моделей, требующих входные тензоры фиксированной размерности. Для решения этой проблемы применяется операция паддинга (дополнения) с помощью функции в коде. Каждая последовательность в обучающем и тестовом наборах обрабатывается в соответствии с ранее заданным параметром. Если отзыв короче этого значения, он дополняется нулями в начале до нужной длины; если длиннее — обрезается с начала, оставляя последние 500 токенов.

```
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)
```

На данном этапе производится формирование финальных выборок для обучения, валидации и тестирования модели в соответствии с заданными ограничениями по размеру. Сначала исходные загруженные данные обрезаются до заранее определённых величин для обучающего и тестового наборов соответственно. Затем из обучающего набора с помощью функции *train\_test\_split* выделяется валидационная выборка, составляющая 20% от его размера. Важно отметить, что разделение происходит с фиксацией случайного состояния, что гарантирует воспроизводимость результатов при повторных запусках. В итоге формируются три независимых набора данных: обучающий — для непосредственного обновления весов модели, валидационный — для подбора гиперпараметров и контроля переобучения, и тестовый — для итоговой объективной оценки качества модели на новых, не участвовавших в настройке данных.

```
x_train, y_train = x_train[:train_size], y_train[:train_size]
x_test, y_test = x_test[:test_size], y_test[:test_size]

x_train, x_val, y_train, y_val = train_test_split(x_train, y_train,
test_size=0.2, random_state=40)

print(f"Размер обучающей выборки: {x_train.shape}")
print(f"Размер валидационной выборки: {x_val.shape}")
print(f"Размер тестовой выборки: {x_test.shape}")
```

#### Вывод:

```
Размер обучающей выборки: (8000, 500)
Размер валидационной выборки: (2000, 500)
Размер тестовой выборки: (1000, 500)
```

Для решения задачи классификации тональности текстов строится нейросетевая модель на основе архитектуры рекуррентной сети. Модель реализована как последовательная (Sequential) и состоит из трёх основных слоёв. Первый слой — Embedding, который преобразует каждый целочисленный индекс слова в плотный вектор фиксированной размерности 32, создавая распределённое векторное представление для всего текста. Этот

слой принимает на вход последовательности длиной, равной 500, и использует словарь с определенным размером. Далее следует рекуррентный слой LSTM с 64 нейронами, который предназначен для обработки последовательностей и захвата контекстных зависимостей между словами в отзыве. На выходе LSTM возвращает только итоговый вектор — финальное скрытое состояние, которое кодирует смысл всего предложения. Финальный полносвязный слой Dense с одним нейроном и сигмоидальной функцией активации преобразует это представление в вероятность принадлежности к классу "положительный отзыв" (вывод близкий к 1) или "отрицательный отзыв" (вывод близкий к 0).

```
model_rnn = keras.Sequential([
    layers.Embedding(input_dim=words_count, output_dim=32,
input_length=maxlen),
    layers.LSTM(64),
    layers.Dense(1, activation='sigmoid')
])

model_rnn.summary()
```

Model: "sequential\_1"

| Layer (type)            | Output Shape | Param #     |
|-------------------------|--------------|-------------|
| embedding_1 (Embedding) | ?            | 0 (unbuilt) |
| lstm_1 (LSTM)           | ?            | 0 (unbuilt) |
| dense_1 (Dense)         | ?            | 0 (unbuilt) |

Total params: 0 (0.00 B)  
Trainable params: 0 (0.00 B)  
Non-trainable params: 0 (0.00 B)

После определения архитектуры модели необходимо сконфигурировать процесс её обучения. Для этого используется метод *compile*, который задаёт ключевые компоненты алгоритма оптимизации. В качестве оптимизатора выбран алгоритм *adam*, являющийся адаптивным и эффективным для широкого класса задач, так как он автоматически настраивает скорость обучения для каждого параметра. Поскольку решается задача бинарной классификации, в качестве функции потерь указана бинарная перекрёстная энтропия, которая хорошо подходит для сравнения предсказанной вероятности (на выходе сигмоиды) с истинной бинарной меткой. Для

мониторинга качества модели в процессе обучения в качестве основной метрики выбрана *accuracy* (доля правильных ответов), которая интуитивно понятна и отражает общую эффективность классификатора.

```
model_rnn.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

На данном этапе осуществляется непосредственно процесс обучения модели RNN. Для этого вызывается метод `fit`, который выполняет итеративное обновление весов сети на протяжении заданного количества эпох. В качестве входных данных используются подготовленные обучающие выборки признаки и метки. Процесс контролируется с помощью валидационной выборки, передаваемой в параметре *validation\_data*. На каждой эпохе модель последовательно обрабатывает данные пакетами размером, равным 128. Параметр `verbose` обеспечивает вывод подробного лога процесса обучения в консоль, отображая прогресс каждой эпохи и значения метрик. Результат выполнения метода сохраняется в объект `history_rnn`, который впоследствии используется для визуализации кривых обучения и анализа динамики метрик.

```
history_rnn = model_rnn.fit(  
    x_train, y_train,  
    epochs=epochs,  
    batch_size=batch_size,  
    validation_data=(x_val, y_val),  
    verbose=1  
)
```

Epoch 1/30

**63/63** ————— **12s** 187ms/step - accuracy: 0.6149  
- loss: 0.6652 - val\_accuracy: 0.7115 - val\_loss: 0.6158

Epoch 2/30

**63/63** ————— **12s** 185ms/step - accuracy: 0.8081  
- loss: 0.4437 - val\_accuracy: 0.8580 - val\_loss: 0.3473

Epoch 3/30

**63/63** ————— **12s** 185ms/step - accuracy: 0.8992  
- loss: 0.2639 - val\_accuracy: 0.8605 - val\_loss: 0.3316

Epoch 4/30

**63/63** ————— **11s** 182ms/step - accuracy: 0.9342  
 - loss: 0.1830 - val\_accuracy: 0.8660 - val\_loss: 0.3405  
 Epoch 5/30

**63/63** ————— **12s** 189ms/step - accuracy: 0.9484  
 - loss: 0.1533 - val\_accuracy: 0.8560 - val\_loss: 0.3731  
 Epoch 6/30

**63/63** ————— **13s** 200ms/step - accuracy: 0.9643  
 - loss: 0.1111 - val\_accuracy: 0.8385 - val\_loss: 0.4185  
 Epoch 7/30

**63/63** ————— **12s** 187ms/step - accuracy: 0.9707  
 - loss: 0.0914 - val\_accuracy: 0.8585 - val\_loss: 0.4771  
 Epoch 8/30

**63/63** ————— **12s** 198ms/step - accuracy: 0.9806  
 - loss: 0.0670 - val\_accuracy: 0.8515 - val\_loss: 0.5145  
 Epoch 9/30

**63/63** ————— **13s** 200ms/step - accuracy: 0.9876  
 - loss: 0.0465 - val\_accuracy: 0.8475 - val\_loss: 0.5095  
 Epoch 10/30

**63/63** ————— **13s** 204ms/step - accuracy: 0.9841  
 - loss: 0.0542 - val\_accuracy: 0.8575 - val\_loss: 0.5419  
 Epoch 11/30

**63/63** ————— **13s** 203ms/step - accuracy: 0.9896  
 - loss: 0.0403 - val\_accuracy: 0.8410 - val\_loss: 0.5661  
 Epoch 12/30

**63/63** ————— **13s** 201ms/step - accuracy: 0.9456  
 - loss: 0.1326 - val\_accuracy: 0.8345 - val\_loss: 0.5373  
 Epoch 13/30

**63/63** ————— **13s** 200ms/step - accuracy: 0.9871  
 - loss: 0.0453 - val\_accuracy: 0.8555 - val\_loss: 0.5898  
 Epoch 14/30

**63/63** ————— **12s** 198ms/step - accuracy: 0.9969  
 - loss: 0.0171 - val\_accuracy: 0.8460 - val\_loss: 0.7060  
 Epoch 15/30

**63/63** ————— **13s** 204ms/step - accuracy: 0.9981  
 - loss: 0.0124 - val\_accuracy: 0.8520 - val\_loss: 0.6834  
 Epoch 16/30

**63/63** ————— **13s** 199ms/step - accuracy: 0.9906  
 - loss: 0.0286 - val\_accuracy: 0.8240 - val\_loss: 0.6239  
 Epoch 17/30

**63/63** ————— **12s** 195ms/step - accuracy: 0.9884  
 - loss: 0.0368 - val\_accuracy: 0.8350 - val\_loss: 0.6250  
 Epoch 18/30

**63/63** ————— **13s** 199ms/step - accuracy: 0.9900  
 - loss: 0.0330 - val\_accuracy: 0.8370 - val\_loss: 0.7101  
 Epoch 19/30

**63/63** ————— **13s** 206ms/step - accuracy: 0.9944  
 - loss: 0.0177 - val\_accuracy: 0.8430 - val\_loss: 0.7515  
 Epoch 20/30

**63/63** ————— **13s** 208ms/step - accuracy: 0.9984  
 - loss: 0.0089 - val\_accuracy: 0.8440 - val\_loss: 0.7900  
 Epoch 21/30

**63/63** ————— **13s** 208ms/step - accuracy: 0.9995  
 - loss: 0.0039 - val\_accuracy: 0.8420 - val\_loss: 0.8635  
 Epoch 22/30

**63/63** ————— **13s** 204ms/step - accuracy: 0.9910  
 - loss: 0.0238 - val\_accuracy: 0.8375 - val\_loss: 0.8325  
 Epoch 23/30



```

63/63 ————— 12s 194ms/step - accuracy: 0.9904
- loss: 0.0271 - val_accuracy: 0.8330 - val_loss: 0.9479
Epoch 24/30
63/63 ————— 13s 204ms/step - accuracy: 0.9865
- loss: 0.0398 - val_accuracy: 0.8425 - val_loss: 0.8077
Epoch 25/30
63/63 ————— 12s 194ms/step - accuracy: 0.9992
- loss: 0.0046 - val_accuracy: 0.8425 - val_loss: 0.8511
Epoch 26/30
63/63 ————— 12s 194ms/step - accuracy: 0.9984
- loss: 0.0062 - val_accuracy: 0.8360 - val_loss: 0.8719
Epoch 27/30
63/63 ————— 13s 200ms/step - accuracy: 0.9895
- loss: 0.0288 - val_accuracy: 0.8395 - val_loss: 0.7386
Epoch 28/30
63/63 ————— 13s 203ms/step - accuracy: 0.9991
- loss: 0.0062 - val_accuracy: 0.8295 - val_loss: 0.8459
Epoch 29/30
63/63 ————— 13s 204ms/step - accuracy: 0.9994
- loss: 0.0050 - val_accuracy: 0.8395 - val_loss: 0.9236
Epoch 30/30
63/63 ————— 13s 205ms/step - accuracy: 0.9999
- loss: 0.0013 - val_accuracy: 0.8420 - val_loss: 0.9879

```

Когда модель была полностью обучилась, произошла проверка, как она справляется с новыми данными. Для этого использовался тестовый набор, который модель раньше не видела. Программа вычислила, сколько ошибок она допустила и какую точность показала.

```

test_loss, test_acc = model_rnn.evaluate(x_test, y_test, verbose=0)
print(f"Точность на тестовых данных: {test_acc:.4f}")

```

## Вывод:

Точность на тестовых данных: 0.8410

Затем были построены графики динамики ключевых метрик. На первом графике отображается изменение функции потерь на обучающей и валидационной выборках в зависимости от номера эпохи. На втором графике представлена динамика точности классификации. Здесь ожидается рост обеих кривых с последующим выходом на плато. Сравнение кривых для обучающей и валидационной данных позволяет визуально оценить, насколько хорошо модель обобщает закономерности: если точность на валидации начинает

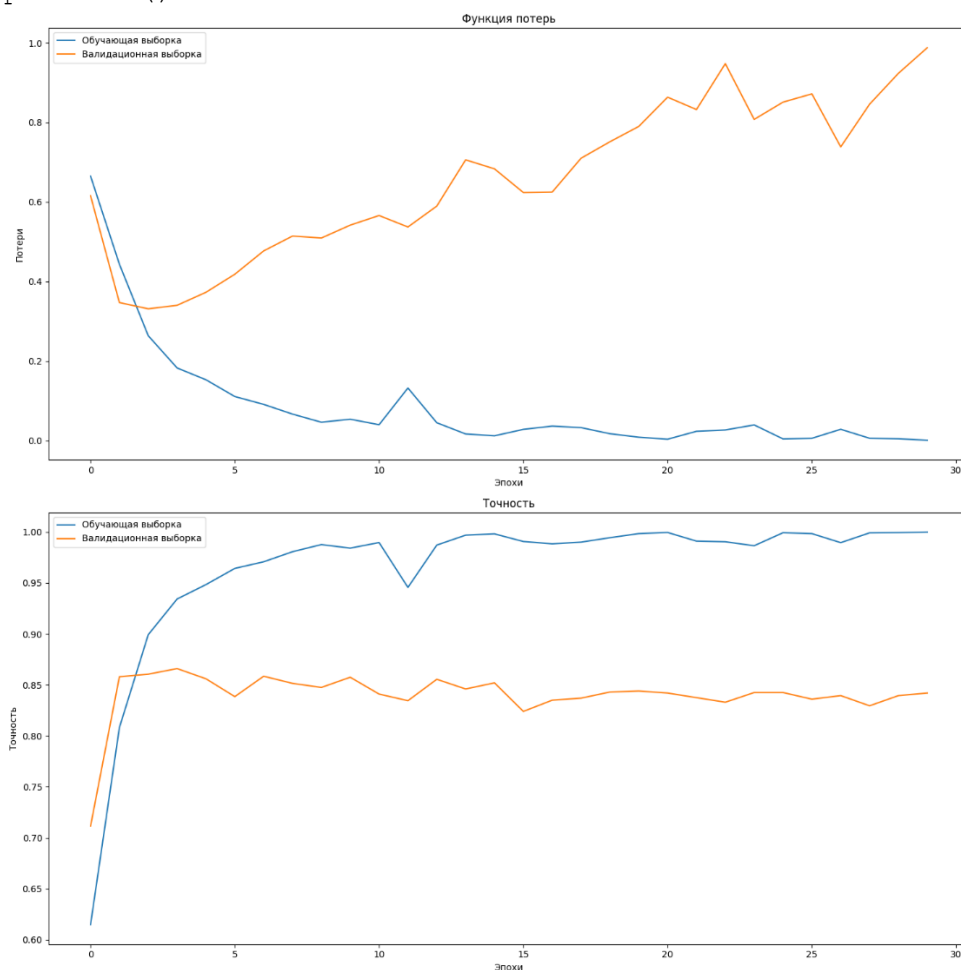
отставать или снижаться при росте точности на обучении, это является признаком переобучения.

```
plt.figure(figsize=(15, 15))
```

```
plt.subplot(2, 1, 1)
plt.plot(history_rnn.history['loss'], label='Обучающая выборка')
plt.plot(history_rnn.history['val_loss'], label='Валидационная выборка')
plt.title('Функция потерь')
plt.xlabel('Эпохи')
plt.ylabel('Потери')
plt.legend()
```

```
plt.subplot(2, 1, 2)
plt.plot(history_rnn.history['accuracy'], label='Обучающая выборка')
plt.plot(history_rnn.history['val_accuracy'], label='Валидационная выборка')
plt.title('Точность')
plt.xlabel('Эпохи')
plt.ylabel('Точность')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



Далее была создана специальная функция. Её задача — взять закодированный отзыв в виде последовательности чисел, преобразовать его обратно в читаемый текст и передать подготовленные данные обученной сети для получения прогноза. Функция выводит на экран последние 300 символов текста отзыва, чтобы можно было понять его содержание. Затем текст проходит через ту же процедуру подготовки, что и все данные ранее — дополняется до стандартной длины. Модель анализирует этот подготовленный вектор и выдаёт число от 0 до 1, интерпретируемое как вероятность положительной оценки. На основе этого порогового значения функция делает итоговый вывод: «позитивный» или «негативный».

```
def predict_sentiment(text_sample_indices):
    """Функция для предсказания эмоциональной окраски"""
    sample_text = decode_review(text_sample_indices)
    print(f"Текст отзыва:\n ...{sample_text[-300:]}")

    sample_processed =
keras.preprocessing.sequence.pad_sequences([text_sample_indices],
maxlen=maxlen)

    prediction = model_rnn.predict(sample_processed, verbose=0)
    sentiment = "Позитивный" if prediction[0][0] > 0.5 else "Негативный"
    print(f"Предсказание: окраска {sentiment.lower()[:-2]+'ая'}")
    print("-" * 50)
    print("\n")

sample_indices = x_test[:4]
for i, sample in enumerate(sample_indices):
    predict_sentiment(sample)
```

## Вывод:

Текст отзыва:

```
...<UNK> and the rest of the cast rendered terrible performances the
show is flat flat flat br br i don't know how michael madison could
have allowed this one on his plate he almost seemed to know this
wasn't going to work out and his performance was quite <UNK> so all
you madison fans give this a miss
```

Предсказание: окраска негативная

-----

Текст отзыва:

```
...er this film is disturbing but it's sincere and it's sure to <UNK>
a strong emotional response from the viewer if you want to see an
```

unusual film some might even say bizarre this is worth the time br br  
unfortunately it's very difficult to find in video stores you may have  
to buy it off the internet  
Предсказание: окраска позитивная  
-----

Текст отзыва:

...choice of material the film stands as a <UNK> tale of universal  
<UNK> <UNK> could be the soviet union italy germany or japan in the  
1930s or any country of any era that lets its guard down and is <UNK>  
by <UNK> it's a fascinating film even a charming one in its macabre  
way but its message is no joke  
Предсказание: окраска позитивная  
-----

Текст отзыва:

...fast forward through everything you see her do until the end also  
is anyone else getting sick of watching movies that are filmed so dark  
anymore one can hardly see what is being filmed as an audience we are  
<UNK> involved with the actions on the screen so then why the hell  
can't we have night vision  
Предсказание: окраска негативная  
-----

Для сравнения результатов была построена альтернативная модель на основе одномерных свёрточных слоёв (CNN). Её архитектура начинается со стандартного слоя векторного представления слов, преобразующего индексы в плотные векторы. Затем следует одномерный свёрточный слой, который с помощью ядра слов скользит по тексту, выявляя локальные шаблоны и сочетания соседних слов. Чтобы получить из выходной последовательности свёртки один вектор на весь отзыв, применяется операция глобального максимума по временной оси. Этот сжатый признаковый вектор проходит через полносвязный скрытый слой с нелинейной функцией активации, после чего к нему применяется регуляризация Drop-out для снижения переобучения: она случайным образом «отключает» половину нейронов во время обучения. Финальный слой с сигмой преобразует полученные признаки в итоговую вероятность положительного отзыва. Модель была скомпилирована с теми же параметрами оптимизатора и функции потерь, что и рекуррентная сеть, для обеспечения корректного сравнения.

```
model_cnn = keras.Sequential([
```

```

        layers.Embedding(input_dim=words_count, output_dim=32,
input_length=maxlen),
        layers.Conv1D(32, 5, activation='relu'),
        layers.GlobalMaxPooling1D(),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ])

model_cnn.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model_cnn.summary()

```

**Model: "sequential\_2"**

| Layer (type)                              | Output Shape | Param #     |
|-------------------------------------------|--------------|-------------|
| embedding_2 (Embedding)                   | ?            | 0 (unbuilt) |
| conv1d (Conv1D)                           | ?            | 0 (unbuilt) |
| global_max_pooling1d (GlobalMaxPooling1D) | ?            | 0           |
| dense_2 (Dense)                           | ?            | 0 (unbuilt) |
| dropout (Dropout)                         | ?            | 0           |
| dense_3 (Dense)                           | ?            | 0 (unbuilt) |

Total params: 0 (0.00 B)  
 Trainable params: 0 (0.00 B)  
 Non-trainable params: 0 (0.00 B)

После настройки архитектуры свёрточной модели был запущен процесс её обучения. Как и в случае с рекуррентной сетью, обучение проводилось в течение тридцати эпох с контролем на валидационной выборке. На каждом шаге данные подавались пакетами по 128 примеров. В ходе этого процесса модель постепенно подстраивала свои внутренние параметры, минимизируя ошибку предсказания на обучающих данных. Параллельно, после каждой эпохи, вычислялись потери и точность на проверочном наборе, что позволяло отслеживать, насколько хорошо сеть обобщает закономерности и не начинает ли она запоминать конкретные примеры. По завершении цикла обучения была выполнена итоговая оценка качества модели на независимом тестовом наборе.

```

history_cnn = model_cnn.fit(
    x_train, y_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_data=(x_val, y_val),
    verbose=1
)

cnn_test_loss, cnn_test_acc = model_cnn.evaluate(x_test, y_test,
verbose=1)

```

## Вывод:

```

Epoch 1/30
63/63 ————— 1s 12ms/step - accuracy: 0.5311 -
loss: 0.6902 - val_accuracy: 0.6355 - val_loss: 0.6778
Epoch 2/30
63/63 ————— 1s 11ms/step - accuracy: 0.7040 -
loss: 0.6067 - val_accuracy: 0.7335 - val_loss: 0.5241
Epoch 3/30
63/63 ————— 1s 11ms/step - accuracy: 0.7994 -
loss: 0.4576 - val_accuracy: 0.7915 - val_loss: 0.4353
Epoch 4/30
63/63 ————— 1s 11ms/step - accuracy: 0.8844 -
loss: 0.3133 - val_accuracy: 0.8425 - val_loss: 0.3608
Epoch 5/30
63/63 ————— 1s 12ms/step - accuracy: 0.9498 -
loss: 0.1776 - val_accuracy: 0.8575 - val_loss: 0.3398
Epoch 6/30
63/63 ————— 1s 12ms/step - accuracy: 0.9811 -
loss: 0.0890 - val_accuracy: 0.8605 - val_loss: 0.3484
Epoch 7/30
63/63 ————— 1s 13ms/step - accuracy: 0.9945 -
loss: 0.0432 - val_accuracy: 0.8540 - val_loss: 0.3559
Epoch 8/30
63/63 ————— 1s 11ms/step - accuracy: 0.9989 -
loss: 0.0225 - val_accuracy: 0.8590 - val_loss: 0.3772
Epoch 9/30
63/63 ————— 1s 12ms/step - accuracy: 0.9994 -
loss: 0.0127 - val_accuracy: 0.8595 - val_loss: 0.4011
Epoch 10/30
63/63 ————— 1s 11ms/step - accuracy: 0.9996 -
loss: 0.0085 - val_accuracy: 0.8540 - val_loss: 0.4304
Epoch 11/30
63/63 ————— 1s 11ms/step - accuracy: 1.0000 -
loss: 0.0062 - val_accuracy: 0.8580 - val_loss: 0.4300
Epoch 12/30
63/63 ————— 1s 11ms/step - accuracy: 0.9999 -
loss: 0.0046 - val_accuracy: 0.8580 - val_loss: 0.4471
Epoch 13/30
63/63 ————— 1s 11ms/step - accuracy: 1.0000 -
loss: 0.0034 - val_accuracy: 0.8595 - val_loss: 0.4501
Epoch 14/30
63/63 ————— 1s 13ms/step - accuracy: 1.0000 -
loss: 0.0028 - val_accuracy: 0.8600 - val_loss: 0.4678

```

```

Epoch 15/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 0.0024 - val_accuracy: 0.8610 - val_loss: 0.4845
Epoch 16/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 0.0021 - val_accuracy: 0.8595 - val_loss: 0.4977
Epoch 17/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 0.0016 - val_accuracy: 0.8570 - val_loss: 0.4926
Epoch 18/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 0.0015 - val_accuracy: 0.8625 - val_loss: 0.5113
Epoch 19/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 0.0013 - val_accuracy: 0.8605 - val_loss: 0.5203
Epoch 20/30
63/63 ----- 1s 11ms/step - accuracy: 1.0000 -
loss: 0.0013 - val_accuracy: 0.8580 - val_loss: 0.5196
Epoch 21/30
63/63 ----- 1s 13ms/step - accuracy: 1.0000 -
loss: 0.0011 - val_accuracy: 0.8580 - val_loss: 0.5264
Epoch 22/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 0.0010 - val_accuracy: 0.8570 - val_loss: 0.5349
Epoch 23/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 8.6508e-04 - val_accuracy: 0.8585 - val_loss: 0.5433
Epoch 24/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 7.9522e-04 - val_accuracy: 0.8590 - val_loss: 0.5607
Epoch 25/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 7.3580e-04 - val_accuracy: 0.8570 - val_loss: 0.5696
Epoch 26/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 5.9603e-04 - val_accuracy: 0.8565 - val_loss: 0.5626
Epoch 27/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 5.9461e-04 - val_accuracy: 0.8580 - val_loss: 0.5718
Epoch 28/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 5.3938e-04 - val_accuracy: 0.8575 - val_loss: 0.5754
Epoch 29/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 4.9893e-04 - val_accuracy: 0.8585 - val_loss: 0.5940
Epoch 30/30
63/63 ----- 1s 12ms/step - accuracy: 1.0000 -
loss: 4.6602e-04 - val_accuracy: 0.8585 - val_loss: 0.5925
32/32 ----- 0s 1ms/step - accuracy: 0.8500 -
loss: 0.6426

```

Далее были выведены итоговые результаты для их сравнения. Для рекуррентной модели была показана точность, полученная на тестовом наборе

ранее. Рядом с ней отобразилась точность, которую продемонстрировала свёрточная сеть после своего обучения и оценки.

```
print(f"RNN модель:")
print(f"Точность на тестовых данных: {test_acc}")

print(f"\nCNN модель:")
print(f"Точность на тестовых данных: {cnn_test_acc}")
```

## Вывод:

RNN модель:  
Точность на тестовых данных: 0.8410000205039978

CNN модель:  
Точность на тестовых данных: 0.8500000238418579

Затем были построены графики. На первом графике изображены кривые точности: сплошные линии показывают, как менялась точность на обучающих данных для RNN и CNN, а пунктирные — соответствующую динамику на валидационной выборке. На втором графике аналогичным образом представлены кривые функции потерь.

```
plt.figure(figsize=(15, 15))

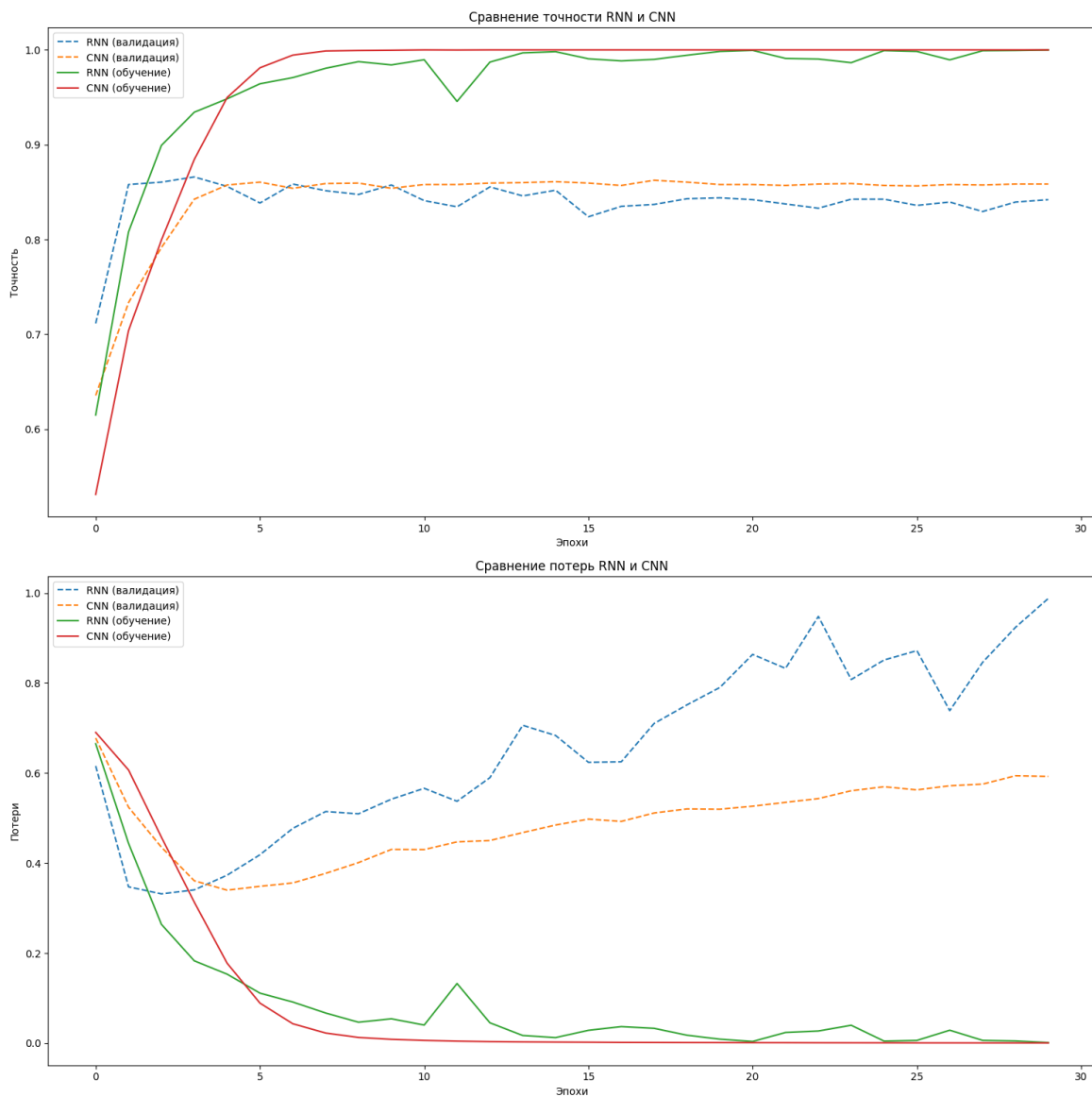
plt.subplot(2, 1, 1)
plt.plot(history_rnn.history['val_accuracy'], label='RNN (валидация)',
         linestyle='--')
plt.plot(history_cnn.history['val_accuracy'], label='CNN (валидация)',
         linestyle='--')
plt.plot(history_rnn.history['accuracy'], label='RNN (обучение)')
plt.plot(history_cnn.history['accuracy'], label='CNN (обучение)')
plt.title('Сравнение точности RNN и CNN')
plt.xlabel('Эпохи')
plt.ylabel('Точность')
plt.legend()

plt.subplot(2, 1, 2)
plt.plot(history_rnn.history['val_loss'], label='RNN (валидация)',
         linestyle='--')
plt.plot(history_cnn.history['val_loss'], label='CNN (валидация)',
         linestyle='--')
plt.plot(history_rnn.history['loss'], label='RNN (обучение)')
plt.plot(history_cnn.history['loss'], label='CNN (обучение)')
```



```
plt.title('Сравнение потерь RNN и CNN')
plt.xlabel('Эпохи')
plt.ylabel('Потери')
plt.legend()

plt.tight_layout()
plt.show()
```



## Вывод:

Хоть рекуррентная модель обучалась и дольше, но она показала худшие результаты по сравнению с одномерной сверточной сетью.

Рекуррентная сеть на первых 3х эпохах обучалась хорошо, потери на валидации падали, а затем начали быстро расти. CNN тоже в какой-то момент начала переобучаться, но делала это медленнее.

## Этап 2. Построение прогноза температуры с использованием рекуррентных и сверточных нейронных сетей.

Импорт библиотек:

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import GRU, Dense, Dropout, Conv1D,
MaxPooling1D, Flatten
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import StandardScaler

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import os
import random
```

Затем задаются ключевые параметры:

```
sequence_length, delay, batch_size = 1440, 144, 64
```

Второй этап начинается с загрузки набора климатических данных. Файл содержит многолетние записи метеорологических параметров, сделанные с фиксированным интервалом. Чтобы сократить время обработки и экспериментов, для анализа берётся только начальный отрезок всего временного ряда — первые триста тысяч измерений.

```
data_path = 'jena_climate_2009_2016.csv'
data = pd.read_csv(data_path)
data = data[:300000]
```

Первые 60% записей выделяются под обучающую выборку. Следующие 20% образуют валидационную выборку, которая будет использоваться для контроля процесса обучения и настройки гиперпараметров. Оставшиеся 20%

данных, самые поздние по времени, формируют тестовую выборку для итоговой оценки качества прогноза модели.

```
train_size = int(0.6 * len(data))
val_size = int(0.2 * len(data))
test_size = len(data) - train_size - val_size
train_data = data[:train_size]
val_data = data[train_size:train_size + val_size]
test_data = data[train_size + val_size:]
```

Для эффективной подачи данных в модель создаётся функция-генератор. Его задача — формировать мини-пакеты (батчи) для обучения. Генератор работает в непрерывном цикле. Каждый раз он случайным образом выбирает в данных множество начальных точек. Для каждой точки формируется последовательность (окно) предыдущих измерений за определённый период — это будут входные признаки модели.

```
def generator(data, lookback=sequence_length, delay=delay,
batch_size=batch_size, step=1):
    while True:
        rows = np.random.randint(1, len(data) - lookback - delay - 1,
size=batch_size)
        samples = np.zeros((batch_size, lookback // step, data.shape[-1]))
        targets = np.zeros((batch_size,))

        for i, row in enumerate(rows):
            indices = range(row, row + lookback, step)
            samples[i] = data.iloc[indices[0]:indices[-1]+1]
            targets[i] = data.iloc[row + lookback + delay - 1][1]

        yield samples, targets
```

Перед обучением модели выполняется подготовка данных. Из всех выборок удаляется столбец с метками времени, так как для прогноза важны только числовые метеорологические параметры. Для обеспечения стабильности и скорости обучения проводится стандартизация данных — каждый признак приводится к нулевому среднему значению и единичному стандартному отклонению. Важно учитывать, что среднее и стандартное отклонение вычисляются только на обучающей выборке, а затем эти же

значения применяются для нормализации валидационных и тестовых данных. Это предотвращает утечку информации из будущего в модель и корректно имитирует условия реального развёртывания, где параметры распределения будущих данных неизвестны. После нормализации на основе каждой из трёх выборок создаётся свой генератор, который будет поставлять модели стандартизированные последовательности в нужном формате.

```
train_data = train_data.drop(columns=['Date Time'])
val_data = val_data.drop(columns=['Date Time'])
test_data = test_data.drop(columns=['Date Time'])

data_mean = train_data.mean()
data_std = train_data.std()
train_data = (train_data - data_mean) / data_std
val_data = (val_data - data_mean) / data_std
test_data = (test_data - data_mean) / data_std

train_generator = generator(train_data, sequence_length, delay,
batch_size)
val_generator = generator(val_data, sequence_length, delay, batch_size)
test_generator = generator(test_data, sequence_length, delay, batch_size)
```

Для решения задачи прогнозирования температуры строится рекуррентная нейронная сеть. Модель состоит из двух последовательных слоёв GRU. Первый слой, содержащий 32 нейрона, настроен на возврат всей выходной последовательности, что необходимо для передачи данных на следующий рекуррентный слой. После него добавляется слой регуляризации Drop-out, который случайным образом отключает 20% нейронов во время обучения для предотвращения переобучения. Второй слой GRU содержит 64 нейрона и возвращает только итоговый вектор — финальное скрытое состояние, которое кодирует информацию из всей входной последовательности. Этот слой также сопровождается Drop-out. Выходной полносвязный слой с одним нейроном без функции активации формирует итоговый прогноз температуры как непрерывную величину.

```
model = Sequential([
```

```

        GRU(32, return_sequences=True, input_shape=(sequence_length,
train_data.shape[-1])),
        Dropout(0.2),
        GRU(64),
        Dropout(0.2),
        Dense(1)
    ])

model.summary()

```

**Model: "sequential"**

| Layer (type)        | Output Shape     | Param # |
|---------------------|------------------|---------|
| gru (GRU)           | (None, 1440, 32) | 4,608   |
| dropout (Dropout)   | (None, 1440, 32) | 0       |
| gru_1 (GRU)         | (None, 64)       | 18,816  |
| dropout_1 (Dropout) | (None, 64)       | 0       |
| dense (Dense)       | (None, 1)        | 65      |

**Total params:** 23,489 (91.75 KB)  
**Trainable params:** 23,489 (91.75 KB)  
**Non-trainable params:** 0 (0.00 B)

Следующим шагом выполняется компиляция модели, то есть задаются параметры, по которым будет происходить её обучение. В качестве оптимизатора выбран алгоритм Adam с фиксированной скоростью обучения, который хорошо зарекомендовал себя для задач регрессии. Поскольку задача заключается в прогнозировании непрерывного значения, в качестве функции потерь выбрано среднее абсолютное отклонение MAE.

```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='mae',
    metrics=['mae']
)

```

Процесс обучения модели запускается на 40 эпохах. На каждой эпохе модель обрабатывает данные, поставляемые обучающим генератором. Количество шагов за эпоху ограничено, что позволяет управлять продолжительностью одного учебного цикла. Параллельно с обучением производится валидация модели: после завершения каждой эпохи она

проверяется на отдельном валидационном генераторе, который подаёт данные, не участвовавшие в обновлении весов. Количество шагов валидации также фиксировано.

```
epochs = 40
validation_steps=200
history = model.fit(
    train_generator,
    epochs=epochs,
    steps_per_epoch=10,
    validation_data=val_generator,
    validation_steps=validation_steps,
    verbose=1
)
```

## Вывод:

```
Epoch 1/40
/var/folders/0g/zw0_s7w5l2zbvj6v0lk842n00000gn/T/ipykernel_96682/1044423498.p
y:10: FutureWarning: Series.__getitem__ treating keys as positions is
deprecated. In a future version, integer keys will always be treated as
labels (consistent with DataFrame behavior). To access a value by position,
use `ser.iloc[pos]`
    targets[i] = data.iloc[row + lookback + delay - 1][1]
10/10 ————— 29s 3s/step - loss: 0.5740 - mae:
0.5740 - val_loss: 0.3362 - val_mae: 0.3362
Epoch 2/40
10/10 ————— 28s 3s/step - loss: 0.3763 - mae:
0.3763 - val_loss: 0.3024 - val_mae: 0.3024
Epoch 3/40
10/10 ————— 27s 3s/step - loss: 0.3431 - mae:
0.3431 - val_loss: 0.3154 - val_mae: 0.3154
Epoch 4/40
10/10 ————— 28s 3s/step - loss: 0.3239 - mae:
0.3239 - val_loss: 0.2953 - val_mae: 0.2953
Epoch 5/40
10/10 ————— 29s 3s/step - loss: 0.3549 - mae:
0.3549 - val_loss: 0.2928 - val_mae: 0.2928
Epoch 6/40
10/10 ————— 28s 3s/step - loss: 0.3330 - mae:
0.3330 - val_loss: 0.2835 - val_mae: 0.2835
Epoch 7/40
10/10 ————— 30s 3s/step - loss: 0.3437 - mae:
0.3437 - val_loss: 0.2777 - val_mae: 0.2777
Epoch 8/40
10/10 ————— 27s 3s/step - loss: 0.3337 - mae:
0.3337 - val_loss: 0.2852 - val_mae: 0.2852
Epoch 9/40
10/10 ————— 30s 3s/step - loss: 0.3343 - mae:
0.3343 - val_loss: 0.2822 - val_mae: 0.2822
Epoch 10/40
10/10 ————— 29s 3s/step - loss: 0.3215 - mae:
0.3215 - val_loss: 0.2910 - val_mae: 0.2910
Epoch 11/40
```

**10/10**  **29s** 3s/step - loss: 0.3278 - mae: 0.3278 - val\_loss: 0.2766 - val\_mae: 0.2766  
 Epoch 12/40

**10/10**  **30s** 3s/step - loss: 0.3117 - mae: 0.3117 - val\_loss: 0.2770 - val\_mae: 0.2770  
 Epoch 13/40

**10/10**  **28s** 3s/step - loss: 0.3417 - mae: 0.3417 - val\_loss: 0.2783 - val\_mae: 0.2783  
 Epoch 14/40

**10/10**  **31s** 3s/step - loss: 0.3258 - mae: 0.3258 - val\_loss: 0.2803 - val\_mae: 0.2803  
 Epoch 15/40

**10/10**  **27s** 3s/step - loss: 0.3203 - mae: 0.3203 - val\_loss: 0.2805 - val\_mae: 0.2805  
 Epoch 16/40

**10/10**  **30s** 3s/step - loss: 0.3276 - mae: 0.3276 - val\_loss: 0.2782 - val\_mae: 0.2782  
 Epoch 17/40

**10/10**  **29s** 3s/step - loss: 0.3152 - mae: 0.3152 - val\_loss: 0.2778 - val\_mae: 0.2778  
 Epoch 18/40

**10/10**  **29s** 3s/step - loss: 0.3334 - mae: 0.3334 - val\_loss: 0.2736 - val\_mae: 0.2736  
 Epoch 19/40

**10/10**  **30s** 3s/step - loss: 0.3157 - mae: 0.3157 - val\_loss: 0.2779 - val\_mae: 0.2779  
 Epoch 20/40

**10/10**  **28s** 3s/step - loss: 0.3067 - mae: 0.3067 - val\_loss: 0.2744 - val\_mae: 0.2744  
 Epoch 21/40

**10/10**  **27s** 3s/step - loss: 0.3029 - mae: 0.3029 - val\_loss: 0.2824 - val\_mae: 0.2824  
 Epoch 22/40

**10/10**  **28s** 3s/step - loss: 0.2964 - mae: 0.2964 - val\_loss: 0.2780 - val\_mae: 0.2780  
 Epoch 23/40

**10/10**  **27s** 3s/step - loss: 0.3132 - mae: 0.3132 - val\_loss: 0.2705 - val\_mae: 0.2705  
 Epoch 24/40

**10/10**  **31s** 3s/step - loss: 0.3242 - mae: 0.3242 - val\_loss: 0.2806 - val\_mae: 0.2806  
 Epoch 25/40

**10/10**  **32s** 3s/step - loss: 0.3181 - mae: 0.3181 - val\_loss: 0.2818 - val\_mae: 0.2818  
 Epoch 26/40

**10/10**  **27s** 3s/step - loss: 0.3020 - mae: 0.3020 - val\_loss: 0.2762 - val\_mae: 0.2762  
 Epoch 27/40

**10/10**  **36s** 4s/step - loss: 0.2971 - mae: 0.2971 - val\_loss: 0.2714 - val\_mae: 0.2714  
 Epoch 28/40

**10/10**  **27s** 3s/step - loss: 0.3116 - mae: 0.3116 - val\_loss: 0.2827 - val\_mae: 0.2827  
 Epoch 29/40

**10/10**  **31s** 3s/step - loss: 0.2983 - mae: 0.2983 - val\_loss: 0.2802 - val\_mae: 0.2802  
 Epoch 30/40

**10/10**  **28s** 3s/step - loss: 0.2972 - mae: 0.2972 - val\_loss: 0.2878 - val\_mae: 0.2878  
 Epoch 31/40

```

10/10 ----- 28s 3s/step - loss: 0.3022 - mae:
0.3022 - val_loss: 0.2821 - val_mae: 0.2821
Epoch 32/40
10/10 ----- 31s 3s/step - loss: 0.3322 - mae:
0.3322 - val_loss: 0.2730 - val_mae: 0.2730
Epoch 33/40
10/10 ----- 31s 3s/step - loss: 0.3065 - mae:
0.3065 - val_loss: 0.2704 - val_mae: 0.2704
Epoch 34/40
10/10 ----- 27s 3s/step - loss: 0.3017 - mae:
0.3017 - val_loss: 0.2784 - val_mae: 0.2784
Epoch 35/40
10/10 ----- 37s 4s/step - loss: 0.3131 - mae:
0.3131 - val_loss: 0.2748 - val_mae: 0.2748
Epoch 36/40
10/10 ----- 33s 3s/step - loss: 0.3032 - mae:
0.3032 - val_loss: 0.2698 - val_mae: 0.2698
Epoch 37/40
10/10 ----- 27s 3s/step - loss: 0.3058 - mae:
0.3058 - val_loss: 0.2752 - val_mae: 0.2752
Epoch 38/40
10/10 ----- 28s 3s/step - loss: 0.3142 - mae:
0.3142 - val_loss: 0.2790 - val_mae: 0.2790
Epoch 39/40
10/10 ----- 30s 3s/step - loss: 0.3024 - mae:
0.3024 - val_loss: 0.2847 - val_mae: 0.2847
Epoch 40/40
10/10 ----- 32s 3s/step - loss: 0.3195 - mae:
0.3195 - val_loss: 0.2723 - val_mae: 0.2723

```

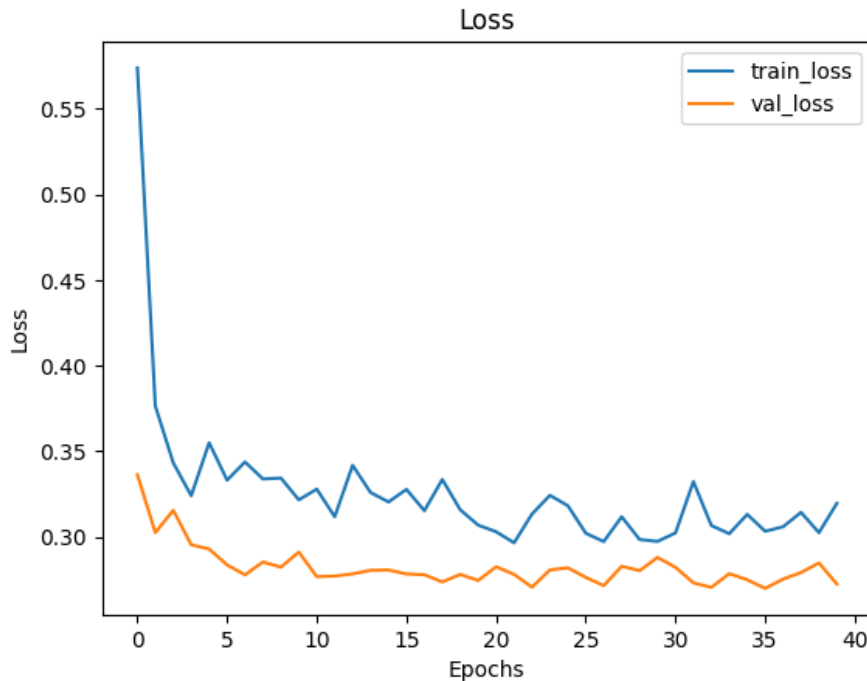
Далее график динамики функции потерь. На нём отображаются две линии: потери на обучающей выборке и потери на валидационной выборке в зависимости от номера эпохи.

```

plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.title('Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

```





Далее создаётся специальная функция. Её задача — получить предсказания на данных из тестового набора, который модель раньше не видела. Функция последовательно берёт пакеты данных из тестового генератора, подаёт их на вход модели и собирает результаты в виде двух списков: предсказанных значений температуры и соответствующих им фактических измерений. Чтобы ограничить объём вычислений, функция получает 100 прогнозов. Поскольку все данные на этапе подготовки были нормализованы, полученные предсказания необходимо преобразовать обратно в исходную шкалу — градусы Цельсия. Для этого используется обратное преобразование с сохранёнными ранее параметрами стандартного отклонения и среднего значения для температуры. После этого строится сравнительный график, на котором отображаются первые 100 точек временного ряда: одна линия показывает реальные значения температуры из тестовой выборки, а вторая — значения, предсказанные рекуррентной моделью.

```
def make_predictions(model, generator, num_predictions=100):
    predictions = []
    actual_values = []

    for x, y in generator:
```

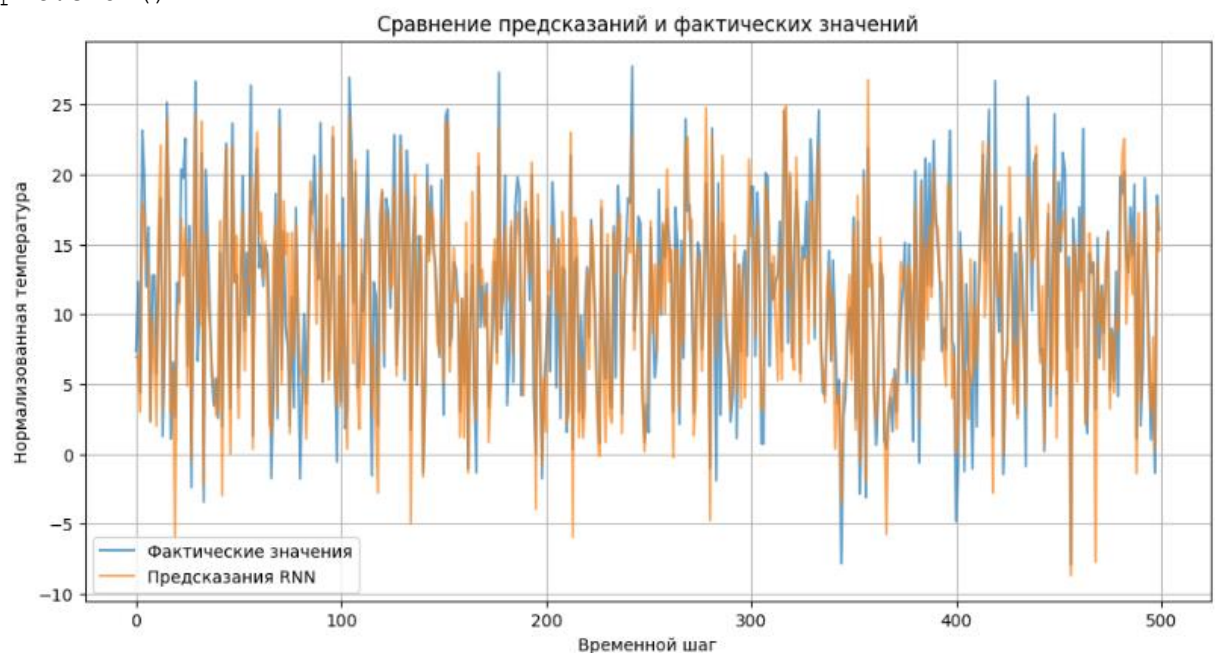
```

    pred = model.predict(x, verbose=0)
    predictions.extend(pred.flatten())
    actual_values.extend(y)
    num_predictions -= 1
    if num_predictions == 0:
        break
    predictions = [x * data_std[1] + data_mean[1] for x in predictions]
    actual_values = [x * data_std[1] + data_mean[1] for x in
actual_values]
    return np.array(predictions), np.array(actual_values)

rnn_predictions, actual_values = make_predictions(model, test_generator)

plt.figure(figsize=(12, 6))
plt.plot(actual_values[:100], label='Фактические значения', alpha=0.7)
plt.plot(rnn_predictions[:100], label='Предсказания RNN', alpha=0.7)
plt.title('Сравнение предсказаний и фактических значений')
plt.xlabel('Временной шаг')
plt.ylabel('Нормализованная температура')
plt.legend()
plt.grid(True)
plt.show()

```



Для сравнения результатов с основной рекуррентной архитектурой строится и обучается гибридная модель, объединяющая подходы свёрточных и рекуррентных сетей. Модель состоит из двух последовательных блоков. Первый блок представляет собой одномерную свёрточную сеть: три слоя свёртки с увеличивающимся количеством фильтров, каждый из которых

следует за слоем подвыборки. Этот блок предназначен для автоматического извлечения локальных временных шаблонов и снижения размерности данных. Второй блок содержит два рекуррентных слоя GRU 32 и 64 нейрона, прореживание 0.2, которые обрабатывают высокоуровневые признаки, извлечённые свёрточным блоком, для улавливания долгосрочных временных зависимостей. Финальный полносвязный слой формирует прогноз температуры. Модель компилируется с теми же параметрами оптимизатора и функцией потерь, что и базовая GRU-модель, что обеспечивает корректность сравнения. Обучение гибридной модели проводится в течение того же количества эпох с использованием тех же генераторов обучающих и валидационных данных.

```
conv_model = Sequential([
    Conv1D(32, 5, activation='relu', input_shape=(sequence_length,
train_data.shape[-1])),
    MaxPooling1D(3),
    Conv1D(64, 5, activation='relu'),
    MaxPooling1D(3),
    Conv1D(128, 5, activation='relu'),
    MaxPooling1D(3),

    GRU(32, return_sequences=True, dropout=0.2, recurrent_dropout=0.2),
    GRU(64, dropout=0.2, recurrent_dropout=0.2),

    Dense(1)
])

conv_model.compile(optimizer='adam', loss='mae', metrics=['mae'])

conv_history = conv_model.fit(
    train_generator, steps_per_epoch=10, epochs=epochs,
    validation_data=val_generator, validation_steps=validation_steps
)
```

## Вывод:

```
Epoch 1/40
/Users/krutukrutzka/github_projects/ML_magister/.venv/lib/python3.10/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```

/var/folders/0g/zw0_s7w512zbvj6v01k842n00000gn/T/ipykernel_68109/1044423498.p
y:10: FutureWarning: Series.__getitem__ treating keys as positions is
deprecated. In a future version, integer keys will always be treated as
labels (consistent with DataFrame behavior). To access a value by position,
use `ser.iloc[pos]`
    targets[i] = data.iloc[row + lookback + delay - 1][1]
10/10  5s 332ms/step - loss: 0.5381 - mae:
0.5381 - val_loss: 0.4659 - val_mae: 0.4659
Epoch 2/40
10/10  3s 312ms/step - loss: 0.4858 - mae:
0.4858 - val_loss: 0.4020 - val_mae: 0.4020
Epoch 3/40
10/10  3s 318ms/step - loss: 0.4577 - mae:
0.4577 - val_loss: 0.4108 - val_mae: 0.4108
Epoch 4/40
10/10  3s 322ms/step - loss: 0.4498 - mae:
0.4498 - val_loss: 0.4326 - val_mae: 0.4326
Epoch 5/40
10/10  3s 316ms/step - loss: 0.4579 - mae:
0.4579 - val_loss: 0.4238 - val_mae: 0.4238
Epoch 6/40
10/10  3s 324ms/step - loss: 0.4413 - mae:
0.4413 - val_loss: 0.3861 - val_mae: 0.3861
Epoch 7/40
10/10  3s 318ms/step - loss: 0.3973 - mae:
0.3973 - val_loss: 0.3780 - val_mae: 0.3780
Epoch 8/40
10/10  3s 323ms/step - loss: 0.4073 - mae:
0.4073 - val_loss: 0.3777 - val_mae: 0.3777
Epoch 9/40
10/10  3s 327ms/step - loss: 0.4068 - mae:
0.4068 - val_loss: 0.3705 - val_mae: 0.3705
Epoch 10/40
10/10  3s 337ms/step - loss: 0.4083 - mae:
0.4083 - val_loss: 0.3539 - val_mae: 0.3539
Epoch 11/40
10/10  3s 322ms/step - loss: 0.3887 - mae:
0.3887 - val_loss: 0.3398 - val_mae: 0.3398
Epoch 12/40
10/10  3s 316ms/step - loss: 0.4362 - mae:
0.4362 - val_loss: 0.4739 - val_mae: 0.4739
Epoch 13/40
10/10  3s 315ms/step - loss: 0.4122 - mae:
0.4122 - val_loss: 0.3750 - val_mae: 0.3750
Epoch 14/40
10/10  3s 312ms/step - loss: 0.3845 - mae:
0.3845 - val_loss: 0.3611 - val_mae: 0.3611
Epoch 15/40
10/10  3s 323ms/step - loss: 0.3886 - mae:
0.3886 - val_loss: 0.3424 - val_mae: 0.3424
Epoch 16/40
10/10  3s 327ms/step - loss: 0.3856 - mae:
0.3856 - val_loss: 0.3528 - val_mae: 0.3528
Epoch 17/40
10/10  3s 320ms/step - loss: 0.3577 - mae:
0.3577 - val_loss: 0.3375 - val_mae: 0.3375
Epoch 18/40
10/10  3s 321ms/step - loss: 0.3648 - mae:
0.3648 - val_loss: 0.3452 - val_mae: 0.3452
Epoch 19/40

```

**10/10**  **3s** 321ms/step - loss: 0.3967 - mae: 0.3967 - val\_loss: 0.3384 - val\_mae: 0.3384  
 Epoch 20/40

**10/10**  **3s** 325ms/step - loss: 0.3705 - mae: 0.3705 - val\_loss: 0.3453 - val\_mae: 0.3453  
 Epoch 21/40

**10/10**  **3s** 325ms/step - loss: 0.3984 - mae: 0.3984 - val\_loss: 0.3206 - val\_mae: 0.3206  
 Epoch 22/40

**10/10**  **3s** 320ms/step - loss: 0.3647 - mae: 0.3647 - val\_loss: 0.3181 - val\_mae: 0.3181  
 Epoch 23/40

**10/10**  **3s** 319ms/step - loss: 0.3526 - mae: 0.3526 - val\_loss: 0.3245 - val\_mae: 0.3245  
 Epoch 24/40

**10/10**  **3s** 318ms/step - loss: 0.3364 - mae: 0.3364 - val\_loss: 0.3262 - val\_mae: 0.3262  
 Epoch 25/40

**10/10**  **3s** 322ms/step - loss: 0.3618 - mae: 0.3618 - val\_loss: 0.3186 - val\_mae: 0.3186  
 Epoch 26/40

**10/10**  **3s** 323ms/step - loss: 0.3576 - mae: 0.3576 - val\_loss: 0.3270 - val\_mae: 0.3270  
 Epoch 27/40

**10/10**  **3s** 324ms/step - loss: 0.3466 - mae: 0.3466 - val\_loss: 0.3238 - val\_mae: 0.3238  
 Epoch 28/40

**10/10**  **3s** 322ms/step - loss: 0.3686 - mae: 0.3686 - val\_loss: 0.3281 - val\_mae: 0.3281  
 Epoch 29/40

**10/10**  **3s** 320ms/step - loss: 0.3392 - mae: 0.3392 - val\_loss: 0.3187 - val\_mae: 0.3187  
 Epoch 30/40

**10/10**  **3s** 322ms/step - loss: 0.3507 - mae: 0.3507 - val\_loss: 0.3201 - val\_mae: 0.3201  
 Epoch 31/40

**10/10**  **3s** 318ms/step - loss: 0.3388 - mae: 0.3388 - val\_loss: 0.3288 - val\_mae: 0.3288  
 Epoch 32/40

**10/10**  **3s** 322ms/step - loss: 0.3302 - mae: 0.3302 - val\_loss: 0.3262 - val\_mae: 0.3262  
 Epoch 33/40

**10/10**  **3s** 327ms/step - loss: 0.3285 - mae: 0.3285 - val\_loss: 0.3259 - val\_mae: 0.3259  
 Epoch 34/40

**10/10**  **3s** 319ms/step - loss: 0.3391 - mae: 0.3391 - val\_loss: 0.3308 - val\_mae: 0.3308  
 Epoch 35/40

**10/10**  **3s** 326ms/step - loss: 0.3513 - mae: 0.3513 - val\_loss: 0.3284 - val\_mae: 0.3284  
 Epoch 36/40

**10/10**  **3s** 334ms/step - loss: 0.3472 - mae: 0.3472 - val\_loss: 0.3184 - val\_mae: 0.3184  
 Epoch 37/40

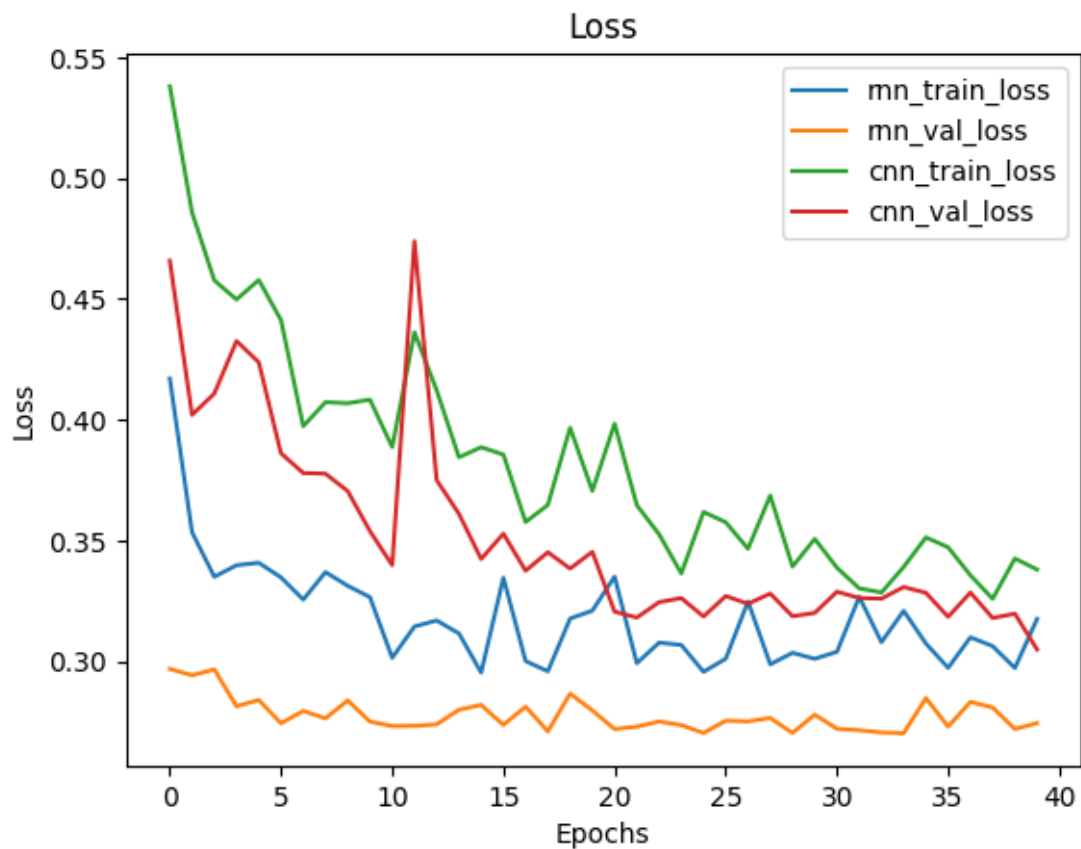
**10/10**  **3s** 324ms/step - loss: 0.3356 - mae: 0.3356 - val\_loss: 0.3285 - val\_mae: 0.3285  
 Epoch 38/40

**10/10**  **3s** 321ms/step - loss: 0.3259 - mae: 0.3259 - val\_loss: 0.3180 - val\_mae: 0.3180  
 Epoch 39/40

```
10/10 ————— 3s 319ms/step - loss: 0.3426 - mae: 0.3426 - val_loss: 0.3197 - val_mae: 0.3197
Epoch 40/40
10/10 ————— 3s 318ms/step - loss: 0.3380 - mae: 0.3380 - val_loss: 0.3049 - val_mae: 0.3049
```

Для объективного сравнения динамики обучения двух моделей — рекуррентной (RNN) и гибридной (CNN+RNN) — строится график изменения функции потерь (MAE) в процессе обучения. На графике отображаются четыре кривые: потери на обучающей и валидационной выборках для каждой из архитектур.

```
plt.plot(history.history['loss'], label='rnn_train_loss')
plt.plot(history.history['val_loss'], label='rnn_val_loss')
plt.plot(conv_history.history['loss'], label='cnn_train_loss')
plt.plot(conv_history.history['val_loss'], label='cnn_val_loss')
plt.title('Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



**Вывод:**

Из графика видно, что рекуррентная сеть показала лучше качество и точнее предсказывала температуру. В то время как смешанная модель обучалась значительно быстрее.

В целом, обе модели показали хороший результат и были достаточно эффективными для задач по прогнозированию временных рядов.

### **Вывод**

В ходе лабораторной работы была изучена концепция рекуррентных нейронных сетей и применены на практике модели LSTM и GRU, а также рекуррентные модели были сопоставлены с одномерной сверточной и смешанной моделями для сравнения эффективности, качества и скорости работы.

В первой задаче нейронная сеть была использована в рамках обработки естественного языка (NLP) для решения задачи бинарной классификации. И RNN, и CNN показали примерно одинаковое качество, но RNN обучалась и предсказывала новые данные значительно медленнее. Это показывает, что для задач такого типа вполне достаточно использовать и более простые модели.

Второй этап заключался в прогнозировании временных рядов. Здесь рекуррентная сеть показала лучше качество по сравнению со смешанной сетью.

В общем, у каждой модели есть свои преимущества и недостатки. У рекуррентных – это более высокая эффективность, а у сверточных сетей – более высокая скорость работы.